

Survey Collector App

Full Project Plan & Technical Documentation

Mobile Application Development | Android (Java)
Final Project — Course Submission

PART 1: Project Overview & System Design

1.1 App Description

Survey Collector is an Android application that allows an Admin to create and manage surveys. Each survey can be shared with respondents via a generated link that opens a lightweight web form. Respondents either fill out the form anonymously or register with a name and email, depending on the survey's configuration. All submitted responses are stored in the cloud and synced back to the Android app, where the Admin can view analytics and manage survey data.

1.2 Target Audience

- Administrators who need to quickly design and distribute surveys
- Respondents who receive a link and fill out a form — no app installation required

1.3 Key Features

- Admin login with session management
- Create, edit, delete, and list surveys
- Add questions of multiple types (text, single-choice, multiple-choice)
- Generate shareable link per survey
- Configure surveys as anonymous or registration-required
- Web-based response form (opened via the shared link)
- Cloud sync of responses via Retrofit + Firebase Realtime Database
- Admin dashboard with basic response analytics
- Offline-first local storage using Room

1.4 Tools & Environment

Component	Technology / Version
Language	Java
Build System	Gradle
Architecture	MVVM (Model-View-ViewModel)
Local Database	Room (SQLite abstraction)
Networking	Retrofit 2 + Gson Converter
Remote API	Firebase Realtime Database (free Spark plan)
UI Framework	Material Design 3

Component	Technology / Version
IDE	Android Studio (recommended) / Cursor with Java+Gradle config
Min SDK	API 24 (Android 7.0)
Target SDK	API 34 (Android 14)

1.5 Scope

In Scope

- Admin authentication (local)
- Full CRUD for Surveys, Questions, and Question Options
- Shareable link generation (deep link or web URL)
- Anonymous and registration-gated response modes
- Cloud sync of responses via Firebase Realtime Database REST API
- Response viewing and simple analytics in the dashboard
- Background sync Service, Broadcast Receiver for connectivity, Content Provider for survey data

Out of Scope

- Multi-admin support or admin roles
- Real-time response notifications (push/WebSocket)
- Advanced analytics or charting beyond basic counts
- File or image uploads within surveys
- Survey expiry or scheduling

1.6 Functional Requirements (CRUD Overview)

Entity	Create	Read	Update	Delete
Survey	Yes	Yes	Yes	Yes
Question	Yes	Yes	Yes	Yes
QuestionOption	Yes	Yes	Yes	Yes
Response	Yes	Yes	No	No
Answer	Yes	Yes	No	No
Respondent	Yes	Yes	No	No
SurveyLink	Yes	Yes	Yes	Yes

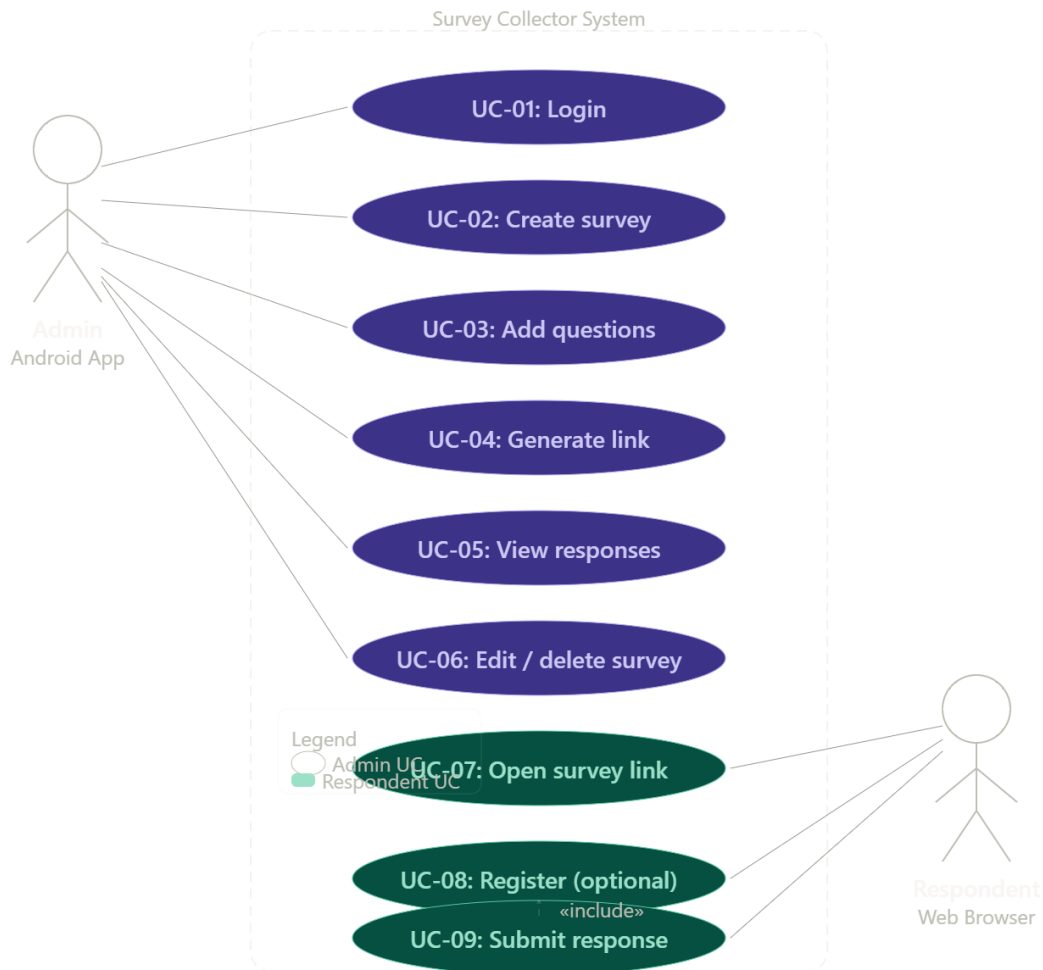
1.7 Actors & Roles

Actor	Access Mode	Permissions
Admin	Android App	Full CRUD on surveys, questions, links; view all responses and analytics; configure survey settings
Respondent	Web Browser	View survey form; optionally register (name + email); submit responses — read-only, submit-only access

1.8 Use Case Summary

The following use cases define actor interactions with the system:

Use Case	Actor	Description
UC-01: Login	Admin	Admin enters credentials and gains access to the app
UC-02: Create Survey	Admin	Admin creates a new survey with title, description, and settings
UC-03: Add Questions	Admin	Admin adds questions (text, single-choice, multiple-choice) to a survey
UC-04: Generate Link	Admin	App generates a unique shareable URL for the survey
UC-05: View Responses	Admin	Admin views submitted responses and basic analytics
UC-06: Edit/Delete Survey	Admin	Admin modifies or removes a survey and its questions
UC-07: Open Survey Link	Respondent	Respondent opens shared link in a browser
UC-08: Register (Optional)	Respondent	If survey requires it, respondent enters name and email
UC-09: Submit Response	Respondent	Respondent fills out and submits the form; data sent to cloud



PART 2: Architecture & Technical Design

2.1 Database Schema — 8 Tables

The local Room database contains 8 tables. All tables mirror the cloud data model used in Firebase Realtime Database, enabling bidirectional sync.

Table 1: Admin

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique admin ID
username	TEXT	NOT NULL, UNIQUE	Login username

Column	Type	Constraints	Description
password	TEXT	NOT NULL	Hashed password
created_at	TEXT	NOT NULL	Account creation timestamp

Table 2: Survey

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique survey ID
admin_id	INTEGER	FOREIGN KEY (Admin.id)	Owner of the survey
title	TEXT	NOT NULL	Survey title
description	TEXT		Optional description
requires_registration	INTEGER	NOT NULL DEFAULT 0	0 = anonymous, 1 = registration required
is_active	INTEGER	NOT NULL DEFAULT 1	Whether the survey is accepting responses
created_at	TEXT	NOT NULL	Creation timestamp

Table 3: Question

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique question ID
survey_id	INTEGER	FOREIGN KEY (Survey.id)	Parent survey
text	TEXT	NOT NULL	Question text
type	TEXT	NOT NULL	TEXT, SINGLE_CHOICE, or MULTIPLE_CHOICE
order_index	INTEGER	NOT NULL DEFAULT 0	Display order within survey

Table 4: QuestionOption

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique option ID
question_id	INTEGER	FOREIGN KEY (Question.id)	Parent question

Column	Type	Constraints	Description
text	TEXT	NOT NULL	Option label text
order_index	INTEGER	NOT NULL DEFAULT 0	Display order

Table 5: Respondent

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique respondent ID
name	TEXT	NOT NULL	Respondent name
email	TEXT	NOT NULL	Respondent email
registered_at	TEXT	NOT NULL	Registration timestamp

Table 6: Response

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique response ID
survey_id	INTEGER	FOREIGN KEY (Survey.id)	The survey this response belongs to
respondent_id	INTEGER	FOREIGN KEY (Respondent.id), NULLABLE	NULL for anonymous responses
submitted_at	TEXT	NOT NULL	Submission timestamp
synced	INTEGER	NOT NULL DEFAULT 0	0 = pending cloud sync, 1 = synced

Table 7: Answer

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique answer ID
response_id	INTEGER	FOREIGN KEY (Response.id)	Parent response
question_id	INTEGER	FOREIGN KEY (Question.id)	The question answered
answer_value	TEXT	NOT NULL	Free text or selected option text

Table 8: SurveyLink

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique link ID
survey_id	INTEGER	FOREIGN KEY (Survey.id)	Linked survey
token	TEXT	NOT NULL, UNIQUE	Random unique token for URL
is_active	INTEGER	NOT NULL DEFAULT 1	Whether the link is still accepting submissions
created_at	TEXT	NOT NULL	Link generation timestamp



2.2 MVVM Architecture Overview

The app follows MVVM (Model-View-ViewModel) architecture for clean separation of concerns:

Layer	Components	Responsibility
View (UI)	Activities, Fragments, XML layouts	Display data, capture user input, observe LiveData
ViewModel	SurveyViewModel, QuestionViewModel, DashboardViewModel, etc.	Hold and prepare UI data, call Repository methods, expose LiveData
Repository	SurveyRepository, ResponseRepository, etc.	Single source of truth; decides between Room (local) and Retrofit (remote)
Room (Local DB)	Entities, DAOs, AppDatabase	Offline-first local storage using SQLite
Retrofit (Remote)	ApiService, ApiClient, DTOs	REST calls to Firebase Realtime Database for cloud sync

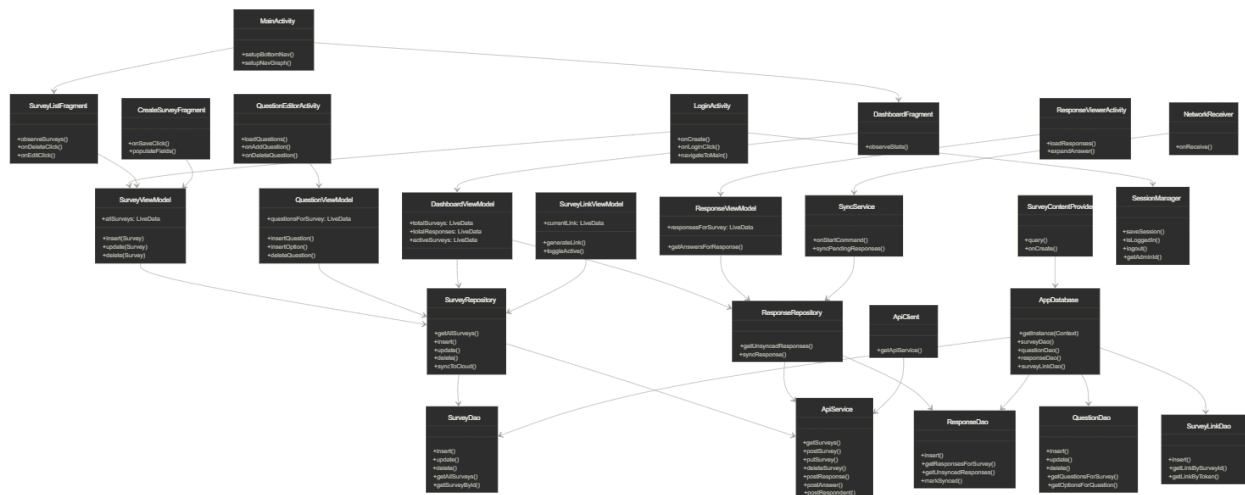
2.3 Screens & Navigation

Screen	Type	Actor	Description
Login Screen	Activity	Admin	Username/password login; stores session in SharedPreferences
Dashboard	Activity	Admin	Shows summary stats: total surveys, total responses, recent activity
Survey List	Fragment	Admin	RecyclerView list of all surveys with edit/delete actions
Create/Edit Survey	Fragment	Admin	Form to create or update a survey; toggle registration requirement
Question Editor	Activity	Admin	Add, reorder, and delete questions and their options
Survey Link Screen	Fragment	Admin	Displays generated link with copy/share button
Response Viewer	Activity	Admin	List of responses per survey with per-question answer breakdown
Web Form (Browser)	Web Page	Respondent	Hosted HTML form; conditionally shows registration fields

2.4 Key Class Structure

Below is a summary of the main classes in each MVVM layer:

Class	Layer	Purpose
LoginActivity	View	Admin login screen; reads session on launch
MainActivity	View	Host for navigation; holds bottom nav or drawer
SurveyListFragment	View	Displays RecyclerView of surveys
CreateSurveyFragment	View	Survey creation/edit form
QuestionEditorActivity	View	Question and option management for a survey
DashboardFragment	View	Shows analytics and summary data
ResponseViewerActivity	View	Displays responses for a selected survey
SurveyViewModel	ViewModel	Manages survey list and CRUD operations via LiveData
QuestionViewModel	ViewModel	Manages questions and options for a given survey
DashboardViewModel	ViewModel	Aggregates stats for the dashboard
ResponseViewModel	ViewModel	Manages loading and displaying responses
SurveyRepository	Repository	Bridges Room DAOs and Retrofit API for surveys
ResponseRepository	Repository	Handles response fetching and sync from cloud
AppDatabase	Room	Room database singleton; registers all DAOs
SurveyDao	Room	CRUD queries for Survey table
QuestionDao	Room	CRUD queries for Question and QuestionOption tables
ResponseDao	Room	Insert and query for Response and Answer tables
ApiService	Retrofit	Retrofit interface defining all REST endpoints
ApiClient	Retrofit	Provides singleton Retrofit instance
SyncService	Service	Background service that syncs unsynced responses to cloud
NetworkReceiver	Broadcast	Listens for connectivity changes; triggers SyncService
SurveyContentProvider	Provider	Exposes survey data via ContentProvider URI interface



2.5 Additional Android Components

Broadcast Receiver — NetworkReceiver

Listens for the `CONNECTIVITY_CHANGE` system broadcast. When the device regains internet connectivity, it starts the `SyncService` to push any locally stored unsynced responses to the cloud. This ensures no data is lost when the respondent submits offline (if in-app) or when the admin re-opens the app.

Service — SyncService

A background Service that queries the Room database for Response rows where `synced = 0`, then POSTs them to Firebase Realtime Database via Retrofit. On success, it updates `synced = 1` for those rows. This runs in the background without blocking the UI thread.

Content Provider — SurveyContentProvider

Exposes Survey and Question data through a standard ContentProvider URI scheme (e.g., `content://com.example.surveycollector/surveys`). This satisfies the project requirement and would allow other apps or system components to query survey data without direct database access.

2.6 Firebase Realtime Database Endpoints

The app uses Firebase Realtime Database as the cloud backend. Firebase exposes every node as a native REST endpoint — no separate backend service is required. All paths append `.json` and use the project base URL: `https://[PROJECT-ID]-default-rtdb.firebaseio.com/`

Method	Endpoint	Return Type	Purpose
GET	surveys.json	Map<String, SurveyDto>	Fetch all surveys
POST	surveys.json	FirebaseKey {name}	Create a new survey — returns auto-generated key
PUT	surveys/{key}.json	SurveyDto	Overwrite a survey node
DELETE	surveys/{key}.json	Void	Remove a survey
GET	responses.json	Map<String, ResponseDto>	Fetch all responses
POST	responses.json	FirebaseKey {name}	Store a new response
POST	answers.json	FirebaseKey {name}	Store a single answer
GET	respondents.json	Map<String, RespondentDto>	Fetch all registered respondents
POST	respondents.json	FirebaseKey {name}	Register a new respondent
GET	surveylinks.json	Map<String, SurveyLinkDto>	Fetch all survey links
POST	surveylinks.json	FirebaseKey {name}	Store a generated link
PATCH	surveylinks/{key}.json	SurveyLinkDto	Update link active status

Note on IDs: Firebase uses auto-generated string keys (e.g. -NxK8abc123), not integer IDs. Room entities store these as a nullable firebaseKey String column, populated after the first successful sync.